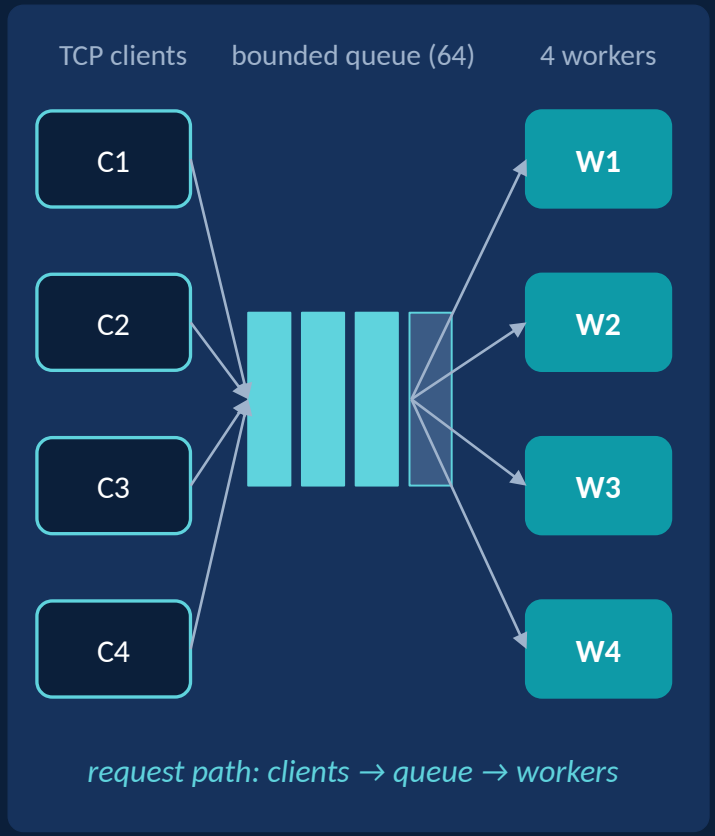
BCSE308P · COMPUTER NETWORKS LAB · FALL SEMESTER 2026–27

Design and Implementation of a Multi-Threaded HTTP/1.1 Web Server from Scratch Using Raw TCP Sockets

Yash Pradhan — 24BCE0702

Anjini Pandey — 24BCE0714

Baseline vs Queue-and-Latency-Aware Admission Control (QLAAC)

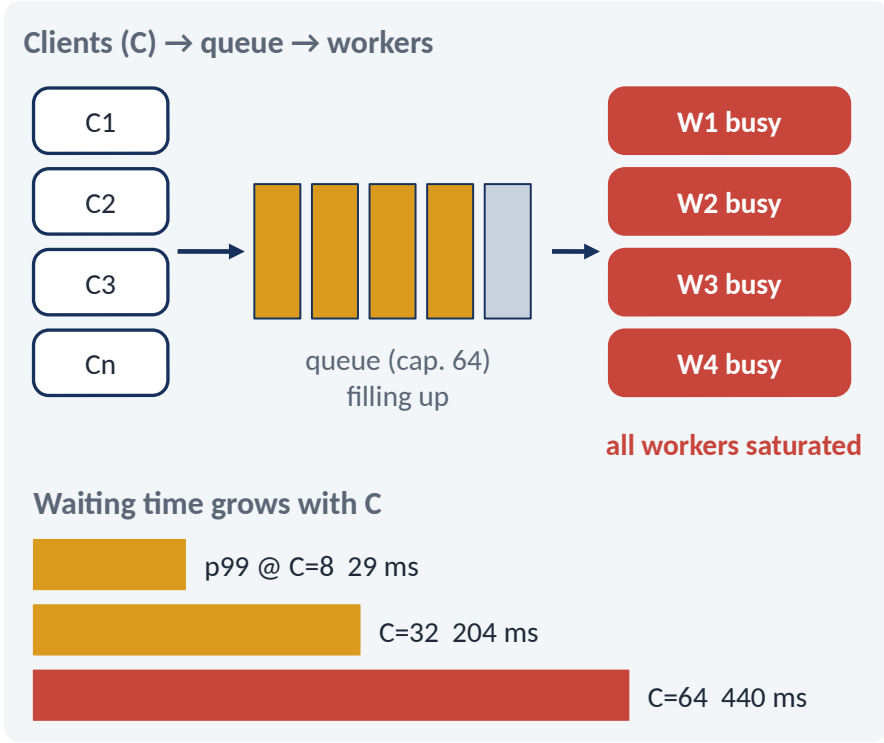


Problem and motivation

- Many clients request service at once
- Server resources are bounded: 4 worker threads, queue of 64
- When all workers are busy, new connections wait in the queue
- Mean latency hides this: p95 / p99 expose the slow tail

Baseline, C = 64

p50 0.52 ms vs p99 440.33 ms



Source: CN_Project_Final_Report (BCSE308P), Sections 1.2, 5.3

Baseline and its gap

Baseline = the same server binary with QLAAC disabled



4 workers, each pinned to one connection (≤100 requests, 5 s idle)

Bottleneck: queue wait behind pinned workers

- Long-lived connections keep workers occupied
- Requests wait behind busy workers as C grows
- Fixed policy: no latency feedback, no early shedding
- Queue-full 503 never triggered (0 5xx in all 20 baseline trials)

Baseline p99 latency of successful responses

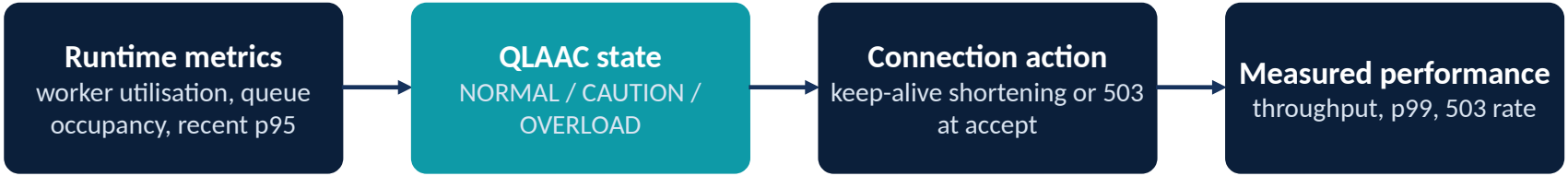
C = 8	C = 32	C = 64
29.09 ms	**204.00 ms**	**440.33 ms**
≈ 6,629 req/s	≈ 6,860 req/s	≈ 6,829 req/s

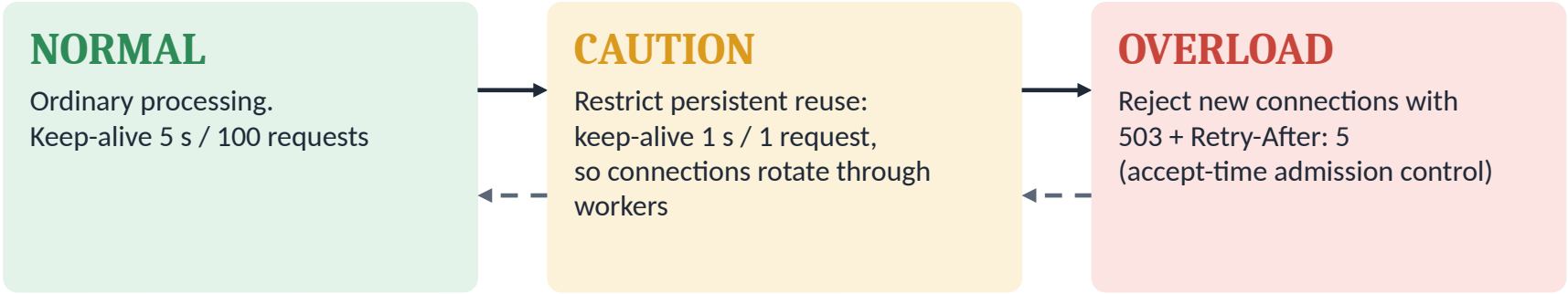
Throughput stays flat while the tail grows.

Source: Final Report §3.2, 5.3, 5.4. Baseline = our own server, QLAAC off; no other server benchmarked.

Proposed novelty: QLAAC

Queue-and-Latency-Aware Admission Control, an in-server feedback controller



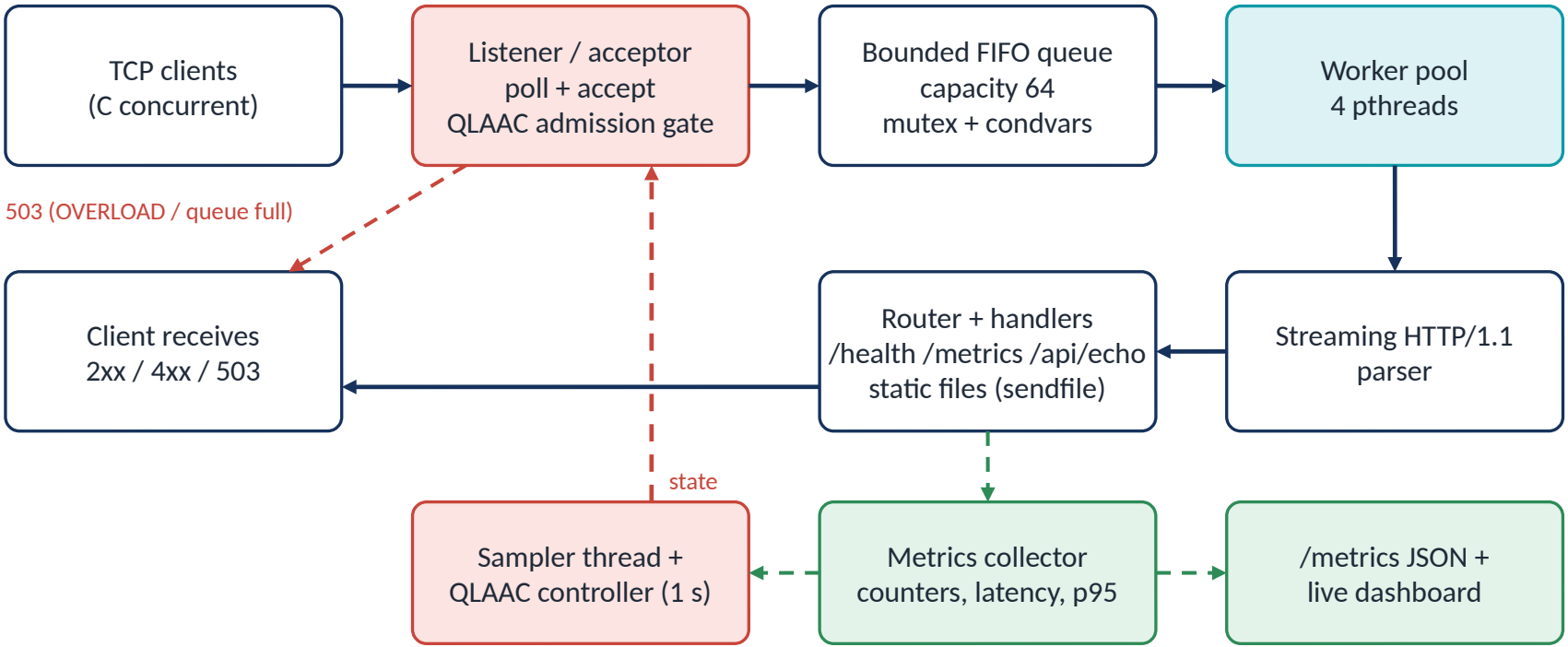


Escalate immediately in one 1-second window · Recover one step after 5 consecutive healthy windows (hysteresis)

Novelty: an in-server controller composing known ideas (feedback admission control, delay-based signals). Not a new standard or algorithm class; the trade-off is what we evaluate.

Source: CN_Project_Final_Report (BCSE308P), Sections 3.3, 3.4 (Fig. 3.2, Table 3.1)

Architecture



Solid: request path · Dashed: feedback and rejection paths

Source: CN_Project_Final_Report (BCSE308P), Section 3.1 (Fig. 3.1), 4.3

Implementation setup

Language / OS
C17, POSIX.1-2008
Linux raw TCP sockets, libc only

Concurrency
pthreads: fixed 4-worker pool, bounded FIFO queue, listen backlog 128

HTTP/1.1 subset
GET, HEAD, POST; keep-alive; Content-Length bodies up to 10 MiB

Routing / files
/health, /metrics, /api/echo (POST), static files via sendfile(2)

Observability
Counters, latency reservoir, /metrics JSON, live dashboard

Verification
72 automated tests (37 C + 35 Python)
Python benchmark harness

Parser limits (implemented)
URI ≤ 2,047 B · ≤ 64 headers · name < 128 B, value < 2,048 B (else 431) · chunked → 501 · Expect: 100-continue → 417

Not implemented / future
TLS, HTTP/2, range requests, chunked bodies, compression, virtual hosts, epoll event loop. Full RFC 9112 compliance is not claimed.

Source: CN_Project_Final_Report (BCSE308P), Sections 1.4, 4.1–4.3 (3,405 lines in src/, 12 files)

Scenarios and metrics

Setup	Value
Concurrency	2, 8, 32, 64 closed-loop clients
Trials	5 × 10-second trials per concurrency/policy condition; 20 trials per policy across four concurrency levels.
Workload	GET /health (89 B), up to 50 requests per keep-alive connection
Server	4 workers, queue 64; QLAAC off vs on (-a)
Environment	Single host, loopback 127.0.0.1:8080, Python load generator
Reporting	Mean ± sample SD (n−1) over the 5 trials

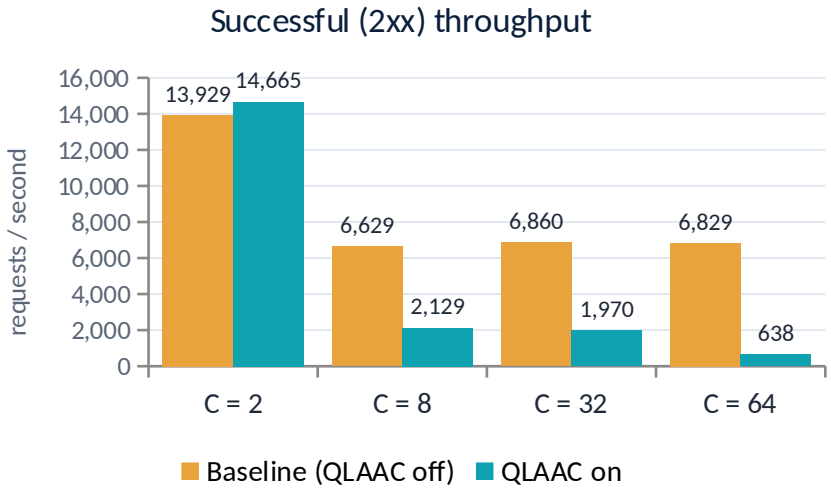
Metrics

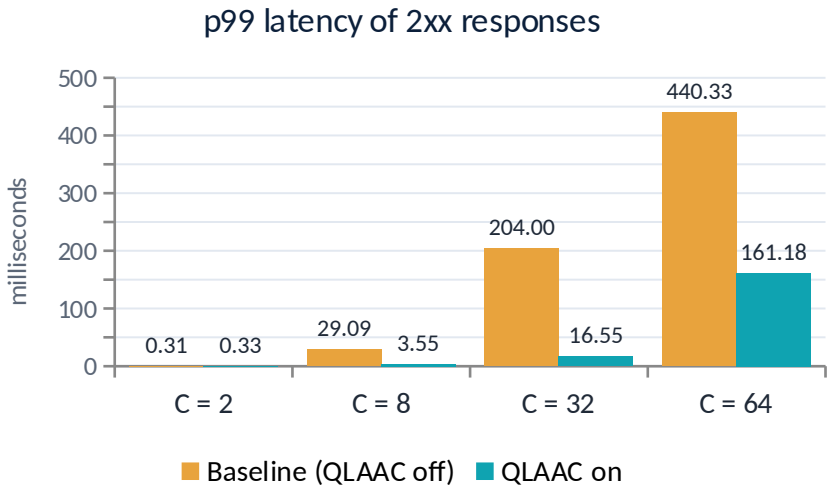
- Successful throughput: 2xx responses per second
- p50 / p95 / p99 latency of 2xx responses (ms)
- HTTP 5xx / 503 count and shedding rate = 5xx ÷ (2xx + 5xx)
- Timeouts and transport errors counted separately

503s and other failures are not counted as throughput. Latency percentiles describe admitted (2xx) requests only, so read them together with the shedding rate.

Source: CN_Project_Final_Report (BCSE308P), Sections 4.2, 5.1, 5.2 (Tables 4.2, 5.1)

Results: baseline vs QLAAC





Change	C=2	C=8	C=32	C=64
Throughput	+5.3% (n.s.)	−67.9%	−71.3%	−90.7%

Change	C=2	C=8	C=32	C=64
p99	≈ same	−87.8%	−91.9%	−63.4%*

QLAAC substantially reduces p99 at higher concurrency, but at a considerable cost in successful throughput. At C = 64, 72.14% ± 5.15% of completed responses were 503.

Source: Final Report Table 5.3 (mean, 5×10 s). *C=64 p99 = 161.18 ± 159.88 ms (high variability). n.s.: not significant.

Analysis and limitations

Key observations

C = 2 — No meaningful difference: C ≤ workers, nothing waits

C = 8, 32 — p99 down 88–92%, throughput down 68–71%, no 503s. Worker saturation alone triggers CAUTION, so every request needs a new connection; p50 rises 0.54 → 1.55 ms and 0.52 → 10.46 ms

C = 64 — 72.14% of completed responses are 503; p99 is unstable (161 ± 160 ms)

Signal gap — Controller p95 starts after dequeue and misses queue wait (≈0.05–0.12 ms at transitions); queue and utilisation drove every transition

Limitations

- Single host, loopback, Python load generator
- One workload (GET /health); 4 levels × 5 trials
- Trials run consecutively without restart: QLAAC state can carry over, so SDs are repeated-trial, not independent
- No controller-state time series recorded
- Thresholds untuned; clients retry 503s at once
- Production generalisation not established

Source: CN_Project_Final_Report (BCSE308P), Sections 4.2, 5.4, 5.5

Conclusion, future work and references

Conclusion

- Built a multi-threaded HTTP/1.1 server on raw TCP sockets (C17, 72 tests)
- Added QLAAC and compared it with the same binary, controller off
- Measured trade-off: much lower p99 at C ≥ 8, but 68–91% less successful throughput

Future work (proposals)

- Measure latency from accept time so queue wait is visible
- Do not throttle on worker saturation alone; scale with queue depth
- Reset state between trials; log controller state
- Open-loop, other workloads, larger or distributed tests

References

[1] R. Fielding, M. Nottingham, J. Reschke, Eds., "HTTP/1.1," IETF RFC 9112 (STD 99), Jun. 2022.

[2] R. Fielding, M. Nottingham, J. Reschke, Eds., "HTTP Semantics," IETF RFC 9110 (STD 97), Jun. 2022.

[5] M. Welsh and D. Culler, "Adaptive overload control for busy Internet servers," in Proc. 4th USENIX USITS, Seattle, WA, Mar. 2003, pp. 45–60.

[11] J. D. C. Little, "A proof for the queuing formula: L = λW," Oper. Res., vol. 9, no. 3, pp. 383–387, Jun. 1961.

References numbered as in the final report (Section 7).